I. Overview

This project is a home security system that sounds an alarm using a buzzer and the device monitors a person's proximity to it using an ultrasonic sensor. It is capable of wireless connection where it displays the system status and allows the user to disarm it over WiFi. The home security system can either be in disarmed mode, armed mode, or alarm mode. Figure 2.1 shows the possible transitions between the different states.

Disarmed mode is when the system is inactive. Armed mode is when the system actively monitors for a person approaching it using an ultrasonic sensor. An onboard LED also blinks in one second intervals in this mode. When a person is in close proximity to the device and does not enter a security code on their phone in time to disarm the system, the home security system goes into alarm mode and sounds the buzzer. However, even if the alarm sounds, the system can still be put back into disarmed mode by entering the security code thereby silencing the alarm. Disarming can also be done with the user push button to represent a manual shut down.

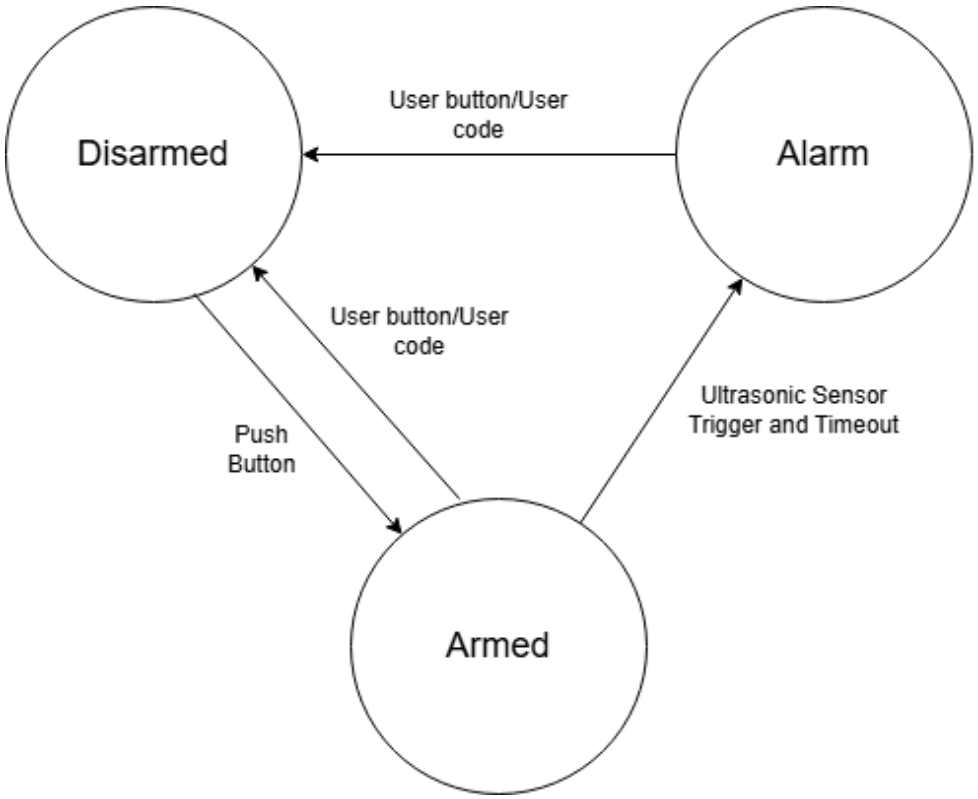


Figure 2.1 Transitions between Disarmed, Armed, and Alarm states

II. Technologies

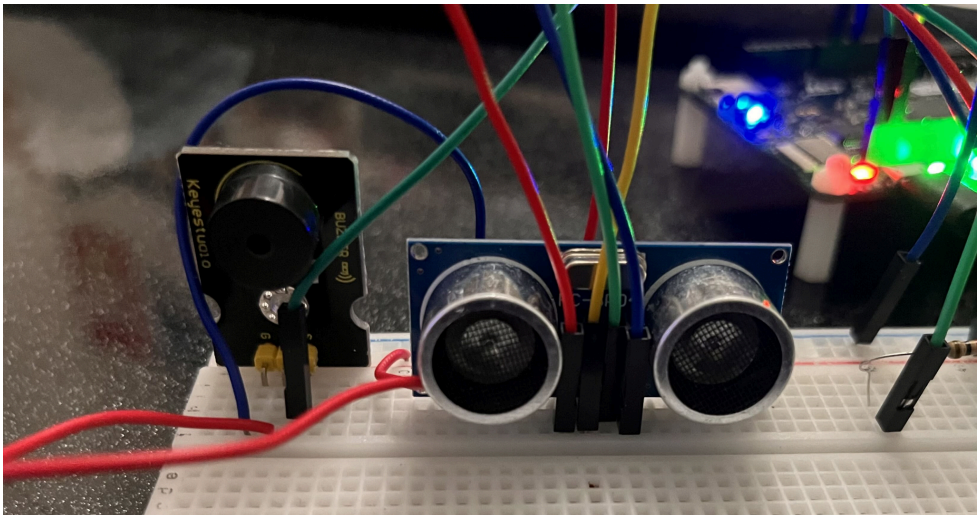


Figure 3.1 Buzzer (left) and HCSR04 ultrasonic sensor (right)

For the project, a buzzer was used and it uses a PWM signal transmitted from the STM board to make a sound. A smaller duty cycle correlates with a lower pitch while a higher duty cycle generates a higher pitch.

The ultrasonic sensor is used to measure the distance between the device and the user. It sends its measurements as digital readings to the microcontroller.

WiFi is used as the connection link between the device and the user. It allows the user to interface the home security system using a mobile device.

III. Implementation

Figure 4.1 shows descriptions of the different tasks used in the Home Security System. Default Task handles the device's WiFi functionalities where the webpage is changed for when the system state changes. It also handles connecting the device to the WiFi network.

Task 02 is the highest priority since it is required to be very responsive to changes in the system. It is supposed to change the system state when prompted and it updates the webpage to display the new system status.

Task 04 handles communication with the ultrasonic sensor in order to get readings on the distance between the device and a user. Readings indicate that the user is too close to the device, Task 04 will signal Task03 to run.

Task03 enforces the Alarm state of the system where it enforces the time period in which the user is expected to enter a code and sounds an alarm if the user is unable to do so.

For task synchronization, I favored using thread flags due to the fact that they are fast and lightweight as they are among the most resource-efficient methods especially in terms of memory.

Task Name	Functionality	Priority
Default Task	Handles transfer of data between server and client over WiFi	osPriorityNormal
Task 02	Changes system state between Disarmed and Armed and updates web page	osPriorityHigh
Task 03	Enforces Alarm mode	osPriorityNormal
Task 04	Initiates trigger input to ultrasonic sensor and checks if the user is too close	osPriorityNormal

Figure 4.1 Descriptions of Home Security System FreeRTOS Tasks

A. Default Task (WiFI Functionality)

Figure 4.2 shows the implementation of the Default Task which calls wifi_server() to initiate a connection between the device and the WiFi network. When a connection is successful, it starts a TCP server and then waits for a client to connect. While waiting for the client, it will advertise the IP address over UART shown in Figure 4.3. When there is a client, the HTTP request is handled and a response is given using the WebServer Process() function shown in FIgure 4.4.

```
/* USER CODE END Header_StartDefaultTask */
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN 5 */
  /* Infinite loop */
  for(;;)
  {
      uint8_t buffer[40] = "Starting wifi in default task";
      HAL_UART_Transmit(&huart1, buffer, sizeof(buffer), 100);
      wifi_server();
      osDelay(2000);
  }
  /* USER CODE END 5 */
}
```

Figure 4.2 Default Task

The WebServerProcess() is important for sending the web page to the user displaying the system status. In Armed mode, the system will also provide a field for the user to enter a code on their device to bypass Alarm mode and shut it down back to Disarmed mode. In order to switch the system back into Disarmed mode, it will change the state to represent Disarmed mode and toggle the timeout flag off. Furthermore, if the buzzer has been ringing, it will also stop the PWM signal controlling it and the timer enforcing the time period for entering the code before Alarm mode is triggered. The resulting webpages created by the Default Task are shown in Figure 4.5.

```
                                    Waiting connection to  192.168.68.
69
  Waiting connection to  192.168.68.69
                                      Waiting connection to  192.168.6
8.69
    Waiting connection to  192.168.68.69
                                        Waiting connection to  192.168
.68.69
      Waiting connection to  192.168.68.69
                                          Waiting connection to  192.1
68.68.69
        Waiting connection to  192.168.68.69
                                            Waiting connection to  192
.168.68.69
          Waiting connection to  192.168.68.69
                                              Waiting connection to  1
92.168.68.69
            Waiting connection to  192.168.68.69
                                                Waiting connection to
 192.168.68.69
              Waiting connection to  192.168.68.69
                                                  Waiting connection t
o  192.168.68.69
                Waiting connection to  192.168.68.69
```

Figure 4.3 Device transmitting IP address over UART for client connection

```
static bool WebServerProcess(void)
{
    uint16_t respLen;
    static uint8_t resp[1024];
    bool stopserver = false;

    if(WIFI_ReceiveData(SOCKET, resp, sizeof(resp)-1, &respLen, WIFI_READ_TIMEOUT) != WIFI_STATUS_OK)
        return stopserver;

    if(respLen == 0)
        return stopserver;

    resp[respLen] = 0;
    char *request = (char*)resp;

    if(strstr(request, "GET") != NULL)
    {
        SendWebPage(state);
    }
    else if(strstr(request, "POST") != NULL)
    {
        char *body = strstr(request, "\r\n\r\n");
        if (body)
        {
            body += 4;

            char pin[7] = {0};
            char *pin_pos = strstr(body, "pin=");
            if (pin_pos)
            {
                pin_pos += 4;
                strncpy(pin, pin_pos, 6);

                if (strcmp(pin, "123456") == 0)
                {
                    state = 0;   // DISARM
                    timed_out = 0;  // Disable timeout
                    HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_1); //Stop PWM
                    HAL_TIM_Base_Stop_IT(&htim7); //Stop One-Shot timer
                    HAL_TIM_Base_Stop_IT(&htim16); //Stop LED timer
                    osThreadFlagsSet(myTask03Handle, FLAG3); //Let the rest of Task 03 run
                }
            }
        }

        SendWebPage(state);
    }

    return stopserver;
}
```

Figure 4.4 WebServerProcess

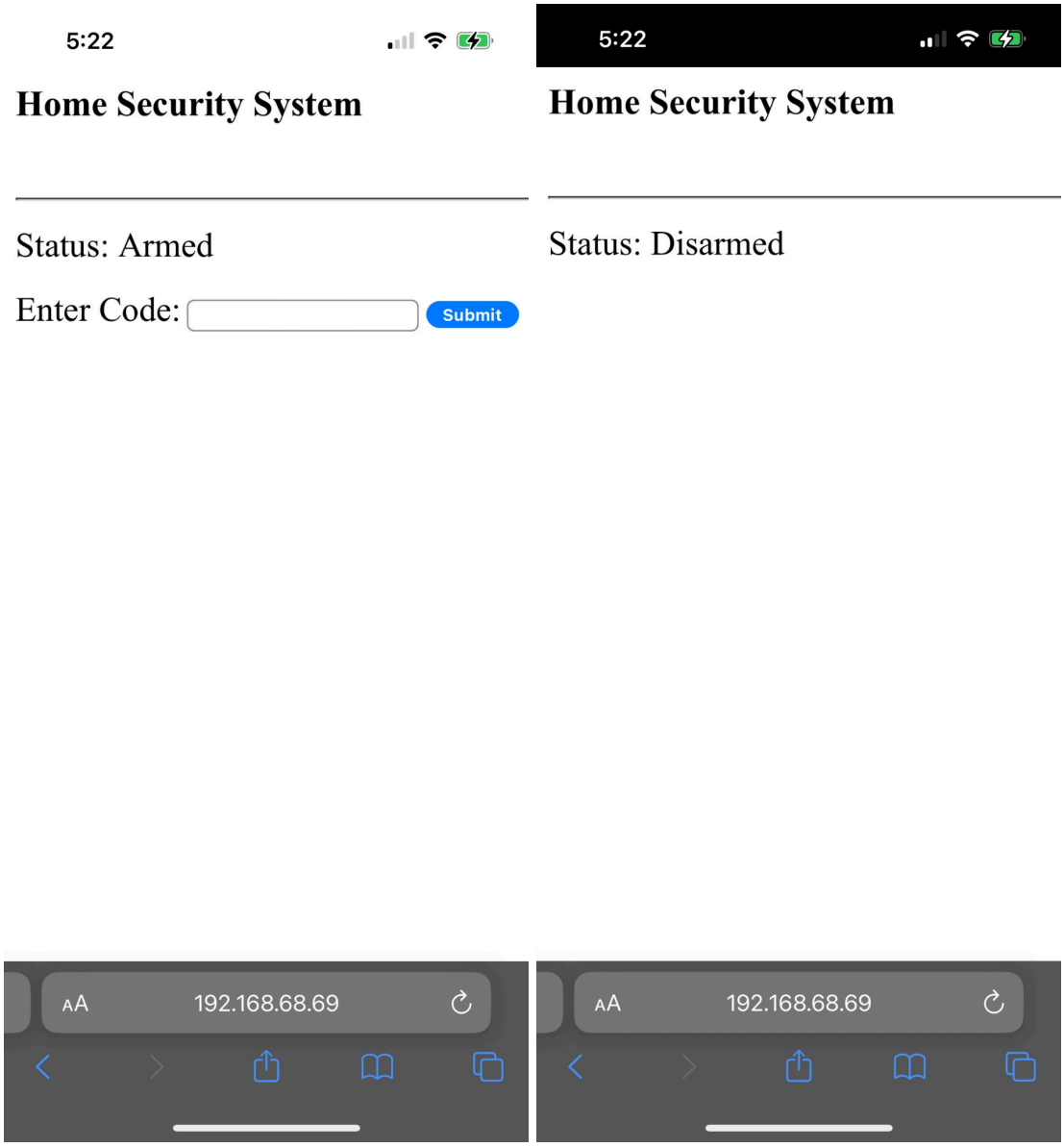


Figure 4.5 Webpages sent by home security system device

B. Task 02 (State Transitions)

```
void StartTask02(void *argument)
{
  /* USER CODE BEGIN StartTask02 */
    // A task that implements a state machine to control the states of the system
    //A task for setting up the timer to control LED delays
  /* Infinite loop */
  for(;;)
  {
      //xSemaphoreTake(sem, portMAX_DELAY);
      osThreadFlagsWait(FLAG4, osFlagsWaitAny, osWaitForever);

      switch(state){
          case 0:
              state = 1;
              HAL_TIM_Base_Start_IT(&htim16);
              break;
          case 1:
              state = 0;
              timed_out = 0;
              HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_1);
              HAL_TIM_Base_Stop_IT(&htim7);
              HAL_TIM_Base_Stop_IT(&htim16);
              break;
          default:
              break;
      }

      SendSSEUpdate();
  }
  /* USER CODE END StartTask02 */
}
```

Figure 4.6 Task 02

Task 02 is responsible for changing the system state between Disarmed and Armed mode. One way to switch the system to Disarmed mode is through sending in the correct code on a device to the system while it is in Armed mode or Alarm mode. The other way is through pressing the push button, which also switches the system from Disarmed mode to Armed mode. This is the case that Task 02 handles.

Figure 4.6 shows the implementation of Task02. Similarly to WebServerProcess(), which is a helper function to Default Task, the task will set the state to represent Disarmed mode and toggle off the timed_out flag. It will also turn off Timer 7 to stop it from enforcing the timeout period for the buzzer and it will turn off Timer 16 used to blink the onboard LED when the system is in Armed mode.

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  /* USER CODE BEGIN Callback 0 */

  /* USER CODE END Callback 0 */
  if (htim->Instance == TIM6)
  {
    HAL_IncTick();
  }
  /* USER CODE BEGIN Callback 1 */
  if (htim->Instance == TIM7)
  {
          timed_out = 1;
          HAL_TIM_Base_Stop_IT(&htim7);
          osThreadFlagsSet(myTask03Handle, FLAG3);

   }
  if (htim->Instance == TIM16){
      HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_14);
  }
  /* USER CODE END Callback 1 */
}
```

Figure 4.7 Callback function for Armed mode LED

Task02 also turns on the LED that blinks once per second when the system is in Armed mode. Timer 16 is used for the LED and the period is enforced by setting the prescaler to 9999 and the ARR register to 9999 as (9999-1)/(80MHz/9999-1) = 1 second. Figure 4.7 shows the callback function called during Timer 16's interrupts. With each interrupt, the LED is toggled once.

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
    {
    switch (GPIO_Pin)
        {
        case (GPIO_PIN_1):
        {
            SPI_WIFI_ISR();
            break;
        }
        case (GPIO_PIN_13):
            //On button press, defer processing to task
            osThreadFlagsSet(myTask02Handle, FLAG4);
            break;
        default:
        {
            break;
        }
    }
}
```

Figure 4.8 Callback function for user button interrupt

However, Task 02 cannot run unless the user presses the user button. Figure 4.8 shows the callback function for when there is an interrupt generated by the button. Here, it defers processing to Task 02 by setting the task's thread flags.

C. Task 04 (Ultrasonic Sensor Measurements)

```
void StartTask04(void *argument)
{
  /* USER CODE BEGIN StartTask04 */
    //A task to determine if somebody has approached the ultrasonic sensor
    uint8_t buffer[20];
  /* Infinite loop */
  for(;;)
  {
    if (state){
        osKernelLock();
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_RESET);
        HAL_Delay(2);

        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_SET);
        HAL_Delay(10);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_RESET);
        osKernelUnlock();

        __HAL_TIM_ENABLE_IT(&htim5, TIM_IT_CC1);

        osThreadFlagsWait(FLAG2, osFlagsWaitAny, osWaitForever);
        snprintf((char*)buffer, sizeof(buffer), "Distance(cm): %u\r\n", (unsigned)Distance);
        HAL_UART_Transmit(&huart1, buffer, sizeof(buffer), HAL_MAX_DELAY);

        if (Distance < 20){
            osThreadFlagsSet(myTask03Handle, FLAG1);
        }
    }

    HAL_Delay(1000);
  }
  /* USER CODE END StartTask04 */
}
```

Figure 4.9 Task 04

Figure 4.9 shows the implementation of Task 04. The task is used for initiating communication with the ultrasonic sensor. It then takes the distance reading it receives from the sensor and transmits the data over UART as shown in Figure 4.10. Furthermore, it uses the data to determine if someone is very close to the device. If this is the case, Task 04 signals Task 03 to run in which it will handle the alarm.

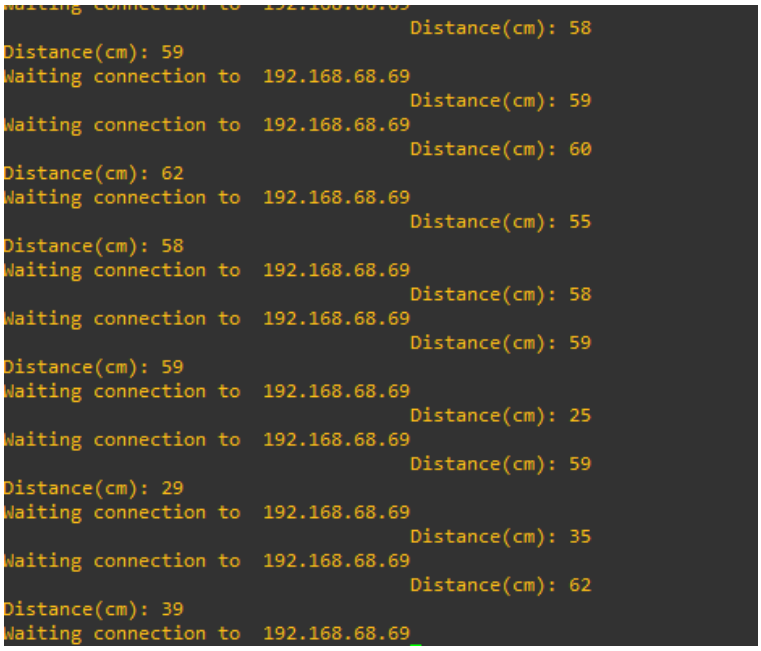


Figure 4.10 Distance readings from ultrasonic sensor

Figure 4.11 shows the sequence of transmissions needed to occur in order to get readings from an ultrasonic sensor. In order to interface with the ultrasonic sensor, Task 04 must send out a 10 microsecond pulse to it first. The ultrasonic sensor will then send out “burst” of sound waves in order to collect a measurement for distance. Lastly, it sends back an echo pulse back to the device in which the pulse duration denotes how far away the object is. A longer pulse will correlate with a greater distance.

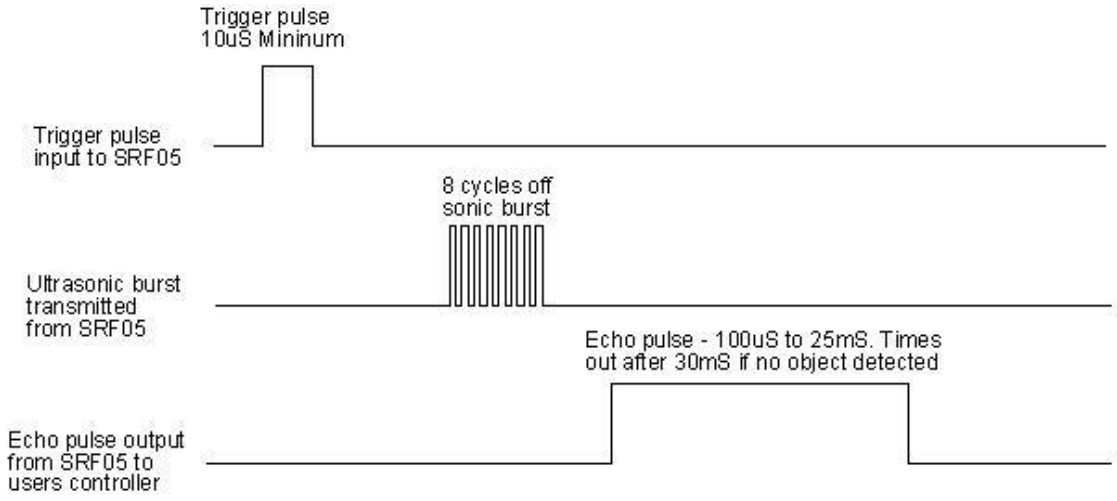


Figure 4.11 Communication sequence with ultrasonic sensor

As shown in Figure 4.9, the kernel is locked from preempting from Task 04 during the trigger pulse transmission since it is a time-sensitive process. Afterwards, it uses Timer 5 for input capture for the ultrasonic sensor's echo pulse where there will be an interrupt when the echo pulse goes high.

Figure 4.12 shows the callback function for these interrupts. On the rising edge of the echo pulse, it will capture the time the pulse went high. Then it waits for an interrupt for when the echo signal falls. At the falling edge, the callback function is executed once again and the timer captures when the signal falls. The later time is subtracted from the earlier time to yield the time reading which is then converted to centimeters.

Then Timer 5 is reconfigured to wait for the rising edge of the echo signal so that it can start reading the next echo pulse. With the distance found, the callback function returns control to Task 04 which will then prompt Task 03 to run if the distance is lower than 20 cm.

```
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2)  // if the interrupt source is channel1
    {
        if (Is_First_Captured==0)
        {
            IC_Val1 = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_2); //First value
            Is_First_Captured = 1;  //First capture
            // Capture at falling edge next time
            __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_2, TIM_INPUTCHANNELPOLARITY_FALLING);
        }

        else if (Is_First_Captured==1)
        {
            IC_Val2 = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_2);  //Second value
            __HAL_TIM_SET_COUNTER(htim, 0);  // reset the counter

            if (IC_Val2 > IC_Val1)
            {
                Difference = IC_Val2-IC_Val1;
            }

            else if (IC_Val1 > IC_Val2)
            {
                Difference = (0xffff - IC_Val1) + IC_Val2;
            }

            Distance = Difference * .034/2;
            Is_First_Captured = 0; //Reset capture count

            // Capture at rising edge next time
            __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_2, TIM_INPUTCHANNELPOLARITY_RISING);
            __HAL_TIM_DISABLE_IT(&htim5, TIM_IT_CC1);

            osThreadFlagsSet(myTask04Handle, FLAG2);
        }
    }
}
```

Figure 4.12 Callback function for Timer 5

D. Task 03 (Alarm Mode)

```
void StartTask03(void *argument)
{
  /* USER CODE BEGIN StartTask03 */
  /* Infinite loop */
  for(;;)
  {
    //When ultrasonic sensor detects, start 10 second one shot timer
    osThreadFlagsWait(FLAG1, osFlagsWaitAny, osWaitForever);

    //Start hardware one shot timer
    __HAL_TIM_CLEAR_FLAG(&htim7, TIM_FLAG_UPDATE);
    __HAL_TIM_SET_COUNTER(&htim7, 0);
    HAL_TIM_Base_Start_IT(&htim7);

    osThreadFlagsWait(FLAG3, osFlagsWaitAny, osWaitForever);

    //If timed out, start buzzer
    if (timed_out && state){
        HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
    }
  }
  /* USER CODE END StartTask03 */
}
```

Figure 4.13 Callback function for Timer 5

Figure 4.13 shows the implementation of Task 03. This task is used for enforcing the period of time where the user is expected to enter the code to disarm the system. If the user doesn't enter the code in time, then the task will start a PWM signal using Timer 2 to sound the buzzer. It only runs when Task 04 signals it to run since the distance captured by the ultrasonic sensor is too low. The task will then start timer 7, which acts as a 10-second one-shot timer to enforce the code entering time duration.

Figure 4.14 shows the callback function for Timer 7. It executes when the ten seconds are up and the system is ready to sound the alarm. Timer 7 toggles the timed_out flag here so that Task 03 starts the PWM signal later on when program control returns to it. Then the timer stops itself as Timer 7 is not intended to be used as a periodic timer. Afterwards, the callback function signals Task 03 to run again by setting its thread flags, where Task 03 can now begin sounding the buzzer with Timer 2.

```c
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  /* USER CODE BEGIN Callback 0 */

  /* USER CODE END Callback 0 */
  if (htim->Instance == TIM6)
  {
    HAL_IncTick();
  }
  /* USER CODE BEGIN Callback 1 */
  if (htim->Instance == TIM7)
  {
          timed_out = 1;
          HAL_TIM_Base_Stop_IT(&htim7);
          osThreadFlagsSet(myTask03Handle, FLAG3);

   }
  if (htim->Instance == TIM16){
      HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_14);
  }
  /* USER CODE END Callback 1 */
}
```

Figure 4.14 Callback function for Timer 5

IV. Conclusion

The home security system was implemented using an STM controller, FreeRTOS, and multiple hardware peripherals. It is designed to be responsive to external events in order to quickly change between Disarmed, Armed, and Alarm modes.

With FreeRTOS, I was able to produce 4 defined tasks with each handling an aspect of functionality of the entire system with the use of priority assignment and interrupts ensuring that the most critical tasks are able to respond quickly. Using thread flags, I was also able to synchronize these tasks.

Timers were used for a variety of tasks. To generate PWM signals, measure digital signals from the ultrasonic sensor, and to perform periodic tasks like toggling the onboard LED. With WiFi, the user is able to interface with the system on their device without need for a keypad.

Using all these technologies, I was able to create a functional, responsive, and interactive device. Further improvements to this device can be that the webpage can be updated in real time without refreshing. Furthermore, it can also send phone notifications so that the system can be armed and disarmed completely remotely by the user and alarms can be monitored.

V. References

1. https://wiki.keyestudio.com/KS0328_Keyestudio_HR-SR04_Blue_Ultrasonic_Module
2. https://controllerstech.com/hcsr04-ultrasonic-sensor-and-stm32/